

Contents

Tech Web, CSS	1
I Fogli di stile	1
Regole di sintassi	1
Come applicare il CSS	2
Regole di applicazione	3
Procedimento a cascata	4
Selettori	4
Selettori di attributo	5
Ereditarietà e specificità	5
Specificità	6
Pseudoclassi	6
Pseudoelementi	7
Sistemi di misura	7
Definizione dei colori	8

Tech Web, CSS

CSS (Cascading Style Sheets), “Cascading” indica che possiamo definire lo stile in diversi punti, e in base a dove lo definiamo ha più o meno priorità. I livelli inferiori prendono lo stile di quelli superiori.

Se si hanno dubbi se una certa funzionalità css è o meno supportata da un browser si può consultare il sito CanIUse.

I Fogli di stile

Insieme di regole che indicano la formattazione da applicare.

Vantaggi: - separazione della grafica dalla struttura - controllo più preciso dell’aspetto grafico - manutenzione più facile - dimensioni ridotte - semplice da capire

Se l’ordine con cui compaiono le cose non dipende più dal layout, da cosa dipende? L’ordine con cui metto gli oggetti dipende dal motore di ricerca, quello che voglio far vedere al motore di ricerca lo metto per primo. In più cerco di strutturare il sito per agevolare gli screen reader.

Regole di sintassi

La sintassi di una regola è: **selettore {blocco di dichiarazioni}** - I selettori sono: - selettori di tag: - `h1 { color:red; font-family: Helvetica, sans-serif; }` - indica che **tutti** gli h1 del documento devono essere rossi e col font Helvetica, se non trova Helvetica usa il font sans serif. - selettori di id - selettori di classe - Blocco di dichiarazioni è un insieme di coppie:

```
{
    proprietà: valore;
    proprietà: valore;
    ...
}
```

Regola per i font: - su dispositivi elettronici si devono usare font senza grazie - su carta stampata si usano font con grazie Questo perché rispettivamente questi font aiutano alla lettura. Alcuni font possono essere più o meno difficili da leggere per le persone con dislessia.

Come applicare il CSS

- Attraverso l'attributo "style" dei tag. NON SI FA MAI, perché non separa contenuto da presentazione.
- Css embedded: si mettono le regole nel tag `<style>` all'interno del `<head>`. Separa un po' meglio, ma non è comunque il massimo perché se voglio usare su più pagine lo stesso stile allora devo ripetere questo codice su ogni pagina.

```
<html>
  <head>
    <style type="text/css">
      h1 {color: red;}
      p {font-family: sans-serif;}
    </style>
  </head>
  ... testo del documento ...
</html>
```

- Fogli di stile esterni: è il modo migliore per dividere contenuto e presentazione. Lo stesso foglio di stile può essere usato su più pagine. È possibile importare più fogli di stile, l'ultimo ha la precedenza.

```
<html>
  <head>
    <link rel="stylesheet" href="stile.css" type="text/css">
  </head>
  ... documento ...
</html>
```

Siccome una pagina può essere visualizzata su più dispositivi possiamo indicare quali fogli di stile usare in base al dispositivo. Per fare ciò si usa l'attributo **media**, che può avere i seguenti valori: - **all**: è di default, applica il foglio a tutti indistintamente. Come tutti i valori di default si può anche non mettere per risparmiare bit - **screen**: per indicare dispositivi con schermo. Per distinguere tra pc e dispositivi mobili si deve fare un controllo ulteriore e si applica il foglio di stile in base alla dimensione dello schermo. - **print**: il foglio di stile che si

applica per quando la pagina viene stampata. - **speech**: c'era una volta per indicare gli screen reader, ma ora non viene più usato.

```
<html>
  <head>
    <title> Esempio con differenti dispositivi </title>
    <link rel="stylesheet" media="screen" href="screen.css" />
    <link rel="stylesheet" media="print" href="print.css" />
    <link rel="stylesheet" media="speech" href="screenreader.css" />
  </head>
  ... documento ...
</html>
```

Per fare il controllo sulla dimensione dello schermo:

```
media="screen and (max-width:480px), only screen and (max-device-width:480px)"
```

La differenza tra **max-width** e **max-device-width** è che **max-width** è la larghezza della finestra, mentre **max-device-width** è la larghezza del dispositivo. Nei cellulari questi due valori sono la stessa cosa, quindi si può lasciare **max-width**. Occhio che magari se sono su pc e rimpicciolisco troppo la finestra non passi alla modalità telefono.

Un altro modo per inserire il css è con **@import**. Sconsigliata.

```
<html>
  <head>
    <style type="text/css">
      @import url("file.css") print;
    </style>
  </head>
  ... testo del documento ...
</html>
```

È una via di mezzo tra un **embedded** e un foglio di stile esterno. Si trova quando si importano font esterni. Un tempo veniva usato per nascondere certe cose a browser molto vecchi, oggi non ha più senso.

Regole di applicazione

Uno stile applicato ad un elemento viene applicato automaticamente anche a tutti i suoi sottoelementi. La regola viene ereditata dai figli a meno che questi non la sovrascrivano.

```
<body style="color: blue">
  <div> ... testo blu... </div>
  <div style="color: green">
    ... testo verde...
  </div>
  <div> ... testo blu... </div>
```

</body>

Procedimento a cascata

Ordine di priorità quando si applicano le regole di stile: 1. Impostazioni personali dell'utente - dimensioni del font o estensioni che modificano la pagina 2. Impostazioni di stile inline definite dall'autore della pagina 3. Fogli di stile embedded definiti dall'autore 4. Fogli di stile esterni definiti dall'autore 5. Impostazioni di stile predefinite del browser - impostazioni utilizzate quando qualcosa non è definito o se il browser non supporta i CSS - cambiano da browser a browser Dall'alto verso il basso è l'ordine di priorità, dal basso verso l'alto è l'ordine di applicazione. QUESTA È UNA DOMANDA D'ESAME.

Per aiutare gli sviluppatori è stata introdotta la definizione di importanza. Se una regola, ovunque si trovi, ha la clausola **!important** davanti questa automaticamente sale al livello 2 di priorità (solo le impostazioni dell'utente la possono sovrascrivere).

```
selettore { proprietà: valore !important; }
```

Quindi l'ordine diventa così:

1. Impostazioni personali dell'utente
2. Dichiarazioni definite con **!important**
3. Impostazioni di stile inline definite dall'autore della pagina
4. Fogli di stile embedded definiti dall'autore
5. Fogli di stile esterni definiti dall'autore
6. Impostazioni di stile predefinite del browser

Generalmente, **!important** viene usato quando non ho capito perché la mia regola di stile non viene visualizzata. Indica quindi che c'è un errore da qualche parte. NON USARLO, la prof lo penalizza.

L'unico caso in cui si può usare è se si sta lavorando con dei framework e quindi non si ha il pieno controllo su tutte le regole applicate.

L'uso dell'**!important** è scoraggiato non solo per una questione di purezza del codice ma anche per via delle performance. Noi vogliamo che il browser renderizzi il più velocemente possibile le nostre pagine. Più è complicato il css, più linee ha, più ci metterà il browser a caricare. Il browser comincia ad applicare regole, quando una regola viene sovrascritta deve tornare indietro e riapplicare tutto. L'**!important** tipicamente fa questo, va a sovrascrivere qualcosa di già caricato e porta il browser a dover fare lavoro in più.

Selettori

Abbiamo detto che la sintassi è **selettore {regola}**. I selettori indicano qual è il tag a cui voglio applicare la mia regola. - Selettori di tipo: si riferiscono all'elemento da formattare - **p { font-size: 1em; }** - Selettori

di attributo: valori degli attributi class e id - Per selezionare una classe:
- `.nomeClasse {font-weight: bold}` - Per selezionare un attributo id: -
`#nomeId {color:blue}`

Il seguente paragrafo avrà tutte i gli stili scritti sopra:

```
<p id="nomeId" class="nomeClasse">Esempio</p>
```

La seguente regola si applica solo ai paragrafi con classe “grande”:

```
p.grande {font-size: 2.5em; }
```

Un tag può avere un’unico id ma può avere più classi. Quando si assegnano le classi al tag si separano da uno spazio.

Se ci sono conflitti viene applicata la regola più specifica. Se il conflitto avviene tra regole con pari specificità viene applicato l’ultimo inserito.

- Selettore universale
 - `* {font-weight: bold;}`
 - si applica a TUTTO
- Raggruppamento di selettori
 - applica le regole a tutti i tag elencati (l’ordine non è rilevante)
 - `h1, h2 {color: blue; font-size: 10pt; }`
- Figli e discendenti
 - l’ordine è rilevante
 - `p em {...}` applica le regole a tutti gli `<em>` contenuti all’interno di un `<p>`, anche se non non c’è discendenza diretta(ovvero se ci sono altri tag in mezzo)
 - se si vuole applicare le regole solo ad attributi figli si fa così:
 - * (esempio) `body > p {...}`
- Selettori di adiacenza
 - `h1 + h2 {...}`
 - * tutti gli `<h2>` che vengono immediatamente dopo un `<h1>`
 - * `<h1>` qui non viene influenzato

Selettori di attributo

Sintassi: `elementname[attributenam=attributevalue]`. Permettono di selezionare un elemento sulla base del valore di un attributo dato.

Esempi: - `abbr[title]` : tutte le abbreviazioni che hanno un attributo `title`, indipendentemente dal valore - `abbr[title="mio titolo"]`: tutte le abbreviazioni con attributo `title` uguale alla stringa “mio titolo” - `abbr[title~="mio titolo"]` : tutte le abbreviazioni con attributo `title` contenente la stringa “mio titolo”

Ereditarietà e specificità

- Ereditarietà : ogni figlio eredita le impostazioni del padre

- Se vengono definite più regole con la stessa importanza per uno stesso elemento, l'ultima definita è quella che verrà applicata (appato che anche la specificità sia la stessa)

Specificità

Oltre all'ordine di priorità menzionato prima: >1. Impostazioni personali dell'utente >2. Dichiarazioni definite con **!important** >3. Impostazioni di stile inline definite dall'autore della pagina >4. Fogli di stile embedded definiti dall'autore >5. Fogli di stile esterni definiti dall'autore >6. Impostazioni di stile predefinite del browser

ricordando che: >Dall'alto verso il basso è l'ordine di priorità, dal basso verso l'alto è l'ordine di applicazione.

aggiungiamo anche il concetto di specificità.

La specificità richiede calcoli piuttosto complessi, che a volte possono essere fonte di errore nei browser.

ATTENZIONE: **solo quando due regole sono in conflitto** la specificità entra in gioco.

Si calcolano tre valori: (*num id, num attributi, num tag html*) (sono in ordine di importanza da sinistra verso destra).

ATTENZIONE: - Le classi sono contate come attributi. - Gli id che sono dentro alle quadre di un attributo non si contano come id ma come attributo.

Si legge il numero a sx, se sono diversi ha priorità quello più alto, se sono uguali si passa al numero in mezzo e si ripete il ragionamento.

```
#nav a {color:orange;} /*(1, 0, 1)*/
a {color:blue;} /*(0, 0, 1)*/
/* In questo caso ha precedenza il primo */
```

In caso di regole che usano la parola **!important** la specificità viene calcolata su 4 valori, in cui la presenza della parola chiave **!important** ha priorità sugli altri.

Pseudoclassi

Non è una classe che io definisco tramite l'attributo **class**, ma è uno stato che un elemento può assumere.

Sintassi: **selettore:pseudoclasse { ... }** - Es: **a:link:hover{ font-size: 2em; }**

PSEUDOCLASSE	RISULTATO
:link	link non visitato
:visited	link visitato
:active	link attivo
:hover	vi si trova sopra il mouse

PSEUDOCLASSE	RISULTATO
:focus	elemento attivo (tab)
:first	prima pag per media paginati
:left	pagine di sinistra
:right	pagine di destra
:first-child	prima occorrenza
:lang	seleziona una lingua

ATTENZIONE: `:hover` funziona SOLO su desktop.

Pseudoelementi

PSEUDOCLASSE	RISULTATO
:first-letter	prima lettera di un blocco
:first-line	prima riga di un blocco
:before	testo da aggiungere prima di un elemento
:after	testo da aggiungere dopo un elemento

Attenzione: in CSS3 possono essere usati per aggiungere del contenuto ma questo contenuto NON è raggiungibile dagli screen reader e dai motori di ricerca.

Sistemi di misura

Esistono diverse unità di misura in CSS, si dividono in: - relative: ex, em, percentuale, rem, vh, vw - assolute: cm, mm, in, pt, px, pc

Si usano SEMPRE quelle relative.

Le unità assolute si usano: - per le media query (tipo vedere quanto è larga una pagina) - per indicare lo spessore di un bordo

Unità	Definizione
em	Altezza media del font utilizzato
px	Numero di pixel nello schermo
in	Inch, pollici (1 in = 2,54 cm)
cm	Centimetri
mm	Millimetri
pt	Punti (1 pt = 1/72 pollici)
pc	Pica (1 pc = 12 punti)
%	Valore in percentuale relativo a quello dell'elemento principale

Definizione dei colori

- Colori predefiniti
 - white, red, green
- Espressi in formato RGB (Red, Green, Blue)
 - #RRGGBB
 - * #FFFFFF è il bianco
 - * se invece di sei numeri ne ho tre, ogni numero si ripete due volte
 - rgb(y,y,y) oppure rgb(y%,y%,y%)
- Colori che funzionano su tutti i browser:
 - aqua
 - black
 - blue
 - fuchsia
 - gray
 - green
 - maroon
 - navy
 - olive
 - purple
 - red
 - silver
 - lime
 - white
 - teal
 - yellow
 - Tutti i colori composti dai codici 00, 33, 66, 99, CC, FF

Nello scritto c'è: - un es su una tabella accessibile - un es sulle regole di specificità
- chiede qual è la regola più specifica e cosa viene applicate - due domande aperte
- delle domande a crocette